

WSL2 기반 VRX + ROS 2 + Simulink 개발환경 구축 가이드 (2026 개정판)

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-03

• Windows 11 · WSL2 · Ubuntu 22.04 · ROS 2 Humble · Gazebo Garden · VRX 2.4.1 · MATLAB R2024b

2025년 「WSL를 이용한 VRX 환경 구축.pptx」의 개정판이다. 원본의 오류 3건을 수정하고, MATLAB ROS Toolbox 연동 절차와 WSL 이미지 배포 절차를 추가했다. 변경 이력은 §10에 정리했다.

0. 시작하기 전에

0.1 하드웨어 요구사항

항목	최소	권장
OS	Windows 10 버전 2004 이상	Windows 11
CPU	4코어 x86-64	i5 / Ryzen 5 이상
RAM	8 GB	16 GB 이상
저장공간	40 GB 여유	SSD
GPU	내장 그래픽	외장 GPU

RAM 8 GB에서는 VRX + Gazebo + MATLAB 동시 구동이 매우 느립니다. 사양 미달 시 1주차 과제 제출 때 신고하면 연구실 워크스테이션 원격 접속을 배정합니다.

0.2 두 가지 설치 경로

경로	소요 시간	대상
A. 배포 이미지 import (§9)	약 20분	대부분의 학생. 권장
B. 처음부터 직접 설치 (§1~§7)	약 2~3시간	전 과정을 이해하고 싶은 학생, 이미지가 안 맞는 경우

- 경로 A로 진행하더라도 §8(MATLAB 연동)과 §1.3(터미널 기본기)은 반드시 수행해야 한다.

0.3 참고 링크

- WSL 설치 — <https://learn.microsoft.com/ko-kr/windows/wsl/install>
- ROS 2 Humble 설치 — <https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html>
- Gazebo Garden 설치 — https://gazebo.org/docs/garden/install_ubuntu/
- VRX 튜토리얼 — <https://github.com/osrf/vrx/wiki/tutorials>
- Mapviz — <https://swri-robotics.github.io/mapviz/>

1. WSL2 + Ubuntu 22.04 설치

1.1 설치

Windows PowerShell을 관리자 권한으로 실행한 뒤:

```
wsl --install -d Ubuntu-22.04
```

⚠ **수정 사항:** 원본 자료의 `wsl --install -d Ubuntu-22.04` 는 하이픈이 유니코드 엔대시(-)여서 실행되지 않는다. 반드시 **ASCII 하이픈(-)** 을 사용할 것.

- 설치 후 재부팅하고, Ubuntu 창이 뜨면 사용자명과 비밀번호를 설정한다.
- 비밀번호는 짧게** 만들 것. `sudo` 실행 때마다 입력해야 한다.
- 비밀번호 입력 시 화면에 아무것도 표시되지 않는 것이 정상이다.** 그대로 타이핑하고 Enter.

1.2 WSL 버전 및 상태 확인

```
wsl -l -v
```

- `VERSION` 이 `2` 인지 확인한다. `1` 이면:

```
wsl --set-version Ubuntu-22.04 2
```

1.3 터미널 실행과 기본 명령어

- Windows Terminal 상단 `+` 옆 화살표 → **Ubuntu-22.04** 선택.

명령어	기능	명령어	기능
<code>cd</code>	경로 이동 (Tab 2회 → 하위 경로 표시)	<code>pwd</code>	현재 경로 출력
<code>ls</code>	파일 목록 (<code>ls -al</code> 상세)	<code>source</code>	스크립트 실행하여 환경 적용
<code>mkdir</code>	디렉토리 생성 (<code>-p</code> 중간경로 포함)	<code>rm</code>	삭제 (<code>-rf</code> 재귀·강제)
<code>cp</code> / <code>mv</code>	복사 / 이동·이름변경	<code>touch</code>	빈 파일 생성
<code>cat</code>	파일 내용 출력	<code>echo</code>	문자열 출력
<code>grep</code>	문자열 검색	<code>chmod</code>	권한 변경
<code>ps</code> / <code>kill</code>	프로세스 조회 / 종료	<code>top</code>	자원 사용량 모니터
<code>df</code> / <code>du</code>	디스크 용량 / 폴더 크기	<code>nano</code>	간단한 텍스트 편집기

팁

- 대부분의 명령어와 경로는 **Tab 키 1~2회로 자동완성**된다. 적극 활용할 것.
- 터미널 복사는 `Ctrl+Shift+C` , 붙여넣기는 `Ctrl+Shift+V` .
- Windows 파일은 `/mnt/c/Users/...` 로 접근 가능하다. 단, **ROS 워크스페이스는 반드시 리눅스 파일시스템 (`~/`)에 두어야 빌드가 빠르다.**

1.4 terminator 설치 (권장)

- 여러 터미널을 동시에 띄워야 하므로 분할 터미널을 설치한다.

```
sudo apt update
sudo apt install -y terminator
terminator
```

단축키	기능	단축키	기능
<code>Ctrl+Alt+T</code>	새 창	<code>Ctrl+Shift+E</code>	수직 분할
<code>Ctrl+Shift+O</code>	수평 분할	<code>Ctrl+Shift+W</code>	현재 분할 닫기
<code>Alt + 방향키</code>	분할 간 이동	<code>Ctrl+Shift+X</code>	현재 분할 최대화 토글

원본 자료의 `Ctrl+Alt+E / O / W` 는 terminator 버전에 따라 동작하지 않는다. Ubuntu 22.04 기본 패키지 기준으로 위 표의 `Ctrl+Shift+*` 조합이 맞다.

이후 모든 작업은 WSL의 Ubuntu 22.04 터미널에서 진행한다.

2. WSL 환경 설정

2.1 리소스 제한 설정 (권장)

- WSL이 Windows 메모리를 과도하게 점유하는 것을 막는다. Windows 쪽 `C:\Users\<사용자명>\.wslconfig` 파일을 만들고:

```
[wsl2]
memory=10GB
processors=4
swap=4GB
```

RAM 16 GB 기준. 8 GB인 경우 `memory=5GB` 로. 저장 후 PowerShell에서 `wsl --shutdown` 실행해 재시작한다.

2.2 GUI 동작 확인 (WSLg)

- Windows 11 및 최신 Windows 10의 WSL2는 WSLg를 내장하므로 별도의 X 서버(VcXsrv 등)가 필요 없다. 확인:

```
sudo apt install -y x11-apps
xeyes
```

- 눈 모양 창이 뜨면 정상이다. 뜨지 않으면 PowerShell에서 `wsl --update` 후 재시도.

3. ROS 2 Humble 설치

3.1 로케일 설정

```
locale # UTF-8 확인

sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

locale # 재확인
```

3.2 저장소 등록

```
sudo apt install -y software-properties-common
sudo add-apt-repository universe

sudo apt update && sudo apt install -y curl
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] ht
tp://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

3.3 패키지 설치

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y ros-humble-desktop
sudo apt install -y python3-colcon-common-extensions python3-rosdep
```

3.4 환경 자동 적용

- 매번 `source` 하지 않도록 `.bashrc` 에 등록한다.

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

3.5 rosdep 초기화

```
sudo rosdep init
rosdep update
```

3.6 설치 확인

- 터미널 두 개를 열고 각각:

```
ros2 run demo_nodes_cpp talker
```

```
ros2 run demo_nodes_py listener
```

- `I heard: [Hello World: 1]` 이 출력되면 성공이다.

4. Gazebo Garden 설치

```
sudo apt update
sudo apt install -y lsb-release curl gnupg

sudo curl https://packages.osrfoundation.org/gazebo.gpg \
  --output /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg] http://packages.osrfoundation.org/gazebo/ubuntu-stable $(lsb_release -cs) main" \
  | sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev/null

sudo apt update
sudo apt install -y gz-garden
```

4.1 ROS 2 연동 패키지

```
sudo apt install -y python3-sdformat13 ros-humble-ros-gz-garden ros-humble-xacro
```

4.2 설치 확인

```
gz sim -v 4 shapes.sdf
```

- Gazebo 창이 뜨고 도형이 보이면 성공. 창이 뜨지 않고 멈추면 §2.2의 WSLg 확인으로 돌아간다.

5. VRX 설치

5.1 클론 및 브랜치 지정

```
mkdir -p ~/vrx_ws/src
cd ~/vrx_ws/src
git clone https://github.com/osrf/vrx.git
cd vrx
git checkout humble          # 필수
cd ~/vrx_ws
```

 **수정 사항 (중요):** 원본 자료에는 `git checkout humble` 단계가 없다. VRX의 기본 브랜치는 최신 조합 (Jazzy + Harmonic)을 따라가므로, 브랜치를 지정하지 않으면 **Gazebo Garden** 환경에서 빌드가 실패하거나 실행 시 플러그인을 못 찾는다.

5.2 의존성 설치 및 빌드

```
cd ~/vrx_ws
rosdep install --from-paths src --ignore-src -r -y
source /opt/ros/humble/setup.bash
colcon build --merge-install
```

빌드에 30~60분 걸린다. RAM이 부족하면 `colcon build --merge-install --parallel-workers 2` 로 병렬도를 낮춘다.

5.3 환경 적용

```
echo "source ~/vrx_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

5.4 실행 확인

```
ros2 launch vrx_gz competition.launch.py world:=sydney_regatta
```

- WAM-V가 시드니 레가타 해역에 떠 있으면 성공이다.

5.5 강의에서 사용하는 월드

upstream VRX 에 원래 들어 있는 월드 — `git clone` 만으로 바로 사용 가능

월드	용도	주차
<code>sydney_regatta</code>	기본 해역, 조종 연습	3~9주
<code>stationkeeping_task</code>	위치유지 과제	13주
<code>wayfinding_task</code>	자세 리스트 추종	9주
<code>perception_task</code>	부표·토탈 인식	12주
<code>gymkhana_task</code>	컬러 게이트 항로 + 장애물	11주
<code>scan_dock_deliver_task</code>	표식 판독 + 도킹	12주

연구실이 추가한 월드 — 배포 파일을 받아 덮어써야 사용 가능

월드	용도	주차
<code>sydney_regatta_ca</code>	적색 마커부표 36개 장애물 필드	10~11주
<code>sydney_regatta_ca_v2</code>	색상 혼합 소규모 필드	10주
<code>scan_dock_deliver_full</code>	도킹 스테이션 3개 베이	12~13주
<code>scan_dock_deliver_CA</code>	부표 80개 + 도킹 플랫폼 (통합)	Term Project

✗ 아래 파일은 `git clone` 으로 받아지지 않는다

- 월드 4개 — `sydney_regatta_ca` · `sydney_regatta_ca_v2` · `scan_dock_deliver_CA` · `scan_dock_deliver_full`
- 런치 파일 — `competition4docking.launch.py`
- 모델 — `custom_docking_station`

전부 연구실이 upstream 에 얹은 파일임. 2026-09-03 기준 osrf/vrx `humble` 브랜치에 없음을 확인함. 10주차 전에 배포 파일을 받아 아래처럼 덮어쓴 뒤 재빌드할 것.

```
# 배포 파일(vrx_overlay.tar.gz)을 받았다고 가정
cd ~
tar -xzf vrx_overlay.tar.gz          # vrx_gz/worlds/, vrx_gz/launch/, vrx_urdf/ 구조로 풀림
cp -r vrx_overlay/* ~/vrx_ws/src/vrx/
cd ~/vrx_ws
colcon build --merge-install
source install/setup.bash
```

- 확인

```
ls ~/vrx_ws/src/vrx/vrx_gz/worlds/ | grep -E "_ca|_full|_CA"
```

- 실행

```
ros2 launch vrx_gz competition.launch.py world:=sydney_regatta_ca
ros2 launch vrx_gz competition4docking.launch.py world:=scan_dock_deliver_full
```

5.6 토픽 확인

```
ros2 topic list
ros2 topic hz /wamv/sensors/gps/gps/fix
ros2 topic echo /wamv/sensors/imu/imu/data --once
```

주요 토픽

토픽	타입	방향
/wamv/sensors/gps/gps/fix	sensor_msgs/NavSatFix	구독
/wamv/sensors/imu/imu/data	sensor_msgs/Imu	구독
/wamv/sensors/lidars/lidar_wamv_sensor/points	sensor_msgs/PointCloud2	구독
/wamv/thrusters/left/thrust	std_msgs/Float64	발행
/wamv/thrusters/right/thrust	std_msgs/Float64	발행
/wamv/thrusters/{left,right}/pos	std_msgs/Float64	(미사용 — 각도 0 고정)

6. Mapviz 설치 (위성지도 항적 표시)

6.1 패키지 설치

```
sudo apt install -y \
  ros-humble-mapviz \
  ros-humble-mapviz-plugins \
  ros-humble-tile-map \
  ros-humble-multires-image
```

⚠ **수정 사항:** 원본 자료의 `ros-$ROS_humble-mapviz` 는 존재하지 않는 변수명이다. 원 튜토리얼의 `$ROS_DISTRO` 를 잘못 치환한 것으로, 이 오타가 원본 슬라이드 18~19의 설치 실패 원인이다. **위와 같이 `ros-humble-` 을 직접 쓸 것.**

6.2 소스 빌드 (플러그인 커스터마이징이 필요한 경우만)

```
cd ~/vrx_ws/src
git clone https://github.com/swri-robotics/mapviz.git
cd ~/vrx_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --merge-install
```

6.3 tf2 관련 오류 발생 시

```
sudo apt update
sudo apt install -y ros-humble-tf2 ros-humble-tf2-ros ros-humble-tf2-geometry-msgs
source /opt/ros/humble/setup.bash
cd ~/vrx_ws
rosdep update
rosdep install --from-paths src --ignore-src -r -y
colcon build --merge-install
```

7. Docker + mapproxy (위성 타일 서버)

7.1 Docker 설치

```
sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl gnupg lsb-release

sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io
```

⚠ 수정 사항 2건:

- 원본은 저장소를 **bionic** (Ubuntu 18.04)으로 지정했다. Ubuntu 22.04에서는 **jammy** 여야 하며, 위치를 **\$(lsb_release -cs)** 로 자동 결정하는 것이 안전하다.
- 원본의 **apt-key add** 는 deprecated 되었다. 위와 같이 **/etc/apt/keyrings/** 에 keyring 파일을 두는 방식으로 대체했다.

7.2 sudo 없이 docker 사용

```
sudo usermod -aG docker $USER
```

- 적용하려면 WSL을 재시작한다 (PowerShell에서 **wsl --shutdown**).

7.3 WSL에서 Docker 데몬 시작

- WSL2에는 systemd가 기본 비활성이므로 수동 시작이 필요할 수 있다.

```
sudo service docker start
sudo service docker status
```

또는 `/etc/wsl.conf` 에 아래를 넣고 `wsl --shutdown` 후 재시작하면 systemd가 활성화되어 `systemctl` 을 쓸 수 있다.

```
[boot]
systemd=true
```

7.4 mapproxy 컨테이너 실행

```
mkdir -p ~/mapproxy
docker run -p 8080:8080 -d -t -v ~/mapproxy:/mapproxy danielsnider/mapproxy
```

7.5 Mapviz launch 파일 수정

- `~/vrx_ws/src/mapviz/mapviz/launch/mapviz.launch.py` 를 아래로 교체한다. 핵심은 **GPS 토픽 리매핑**(`fix` → `/wamv/sensors/gps/gps/fix`)과 `map` → `origin` static TF이다.

```
import launch
import launch_ros.actions

def generate_launch_description():
    return launch.LaunchDescription([
        launch_ros.actions.Node(
            package="mapviz", executable="mapviz", name="mapviz",
        ),
        launch_ros.actions.Node(
            package="swri_transform_util",
            executable="initialize_origin.py",
            name="initialize_origin",
            parameters=[
                {"local_xy_frame": "map"},
                {"local_xy_origin": "auto"},
            ],
            remappings=[('fix', '/wamv/sensors/gps/gps/fix')],
        ),
        launch_ros.actions.Node(
            package="tf2_ros",
            executable="static_transform_publisher",
            name="swri_transform",
            arguments=["0", "0", "0", "0", "0", "0", "map", "origin"],
        ),
    ])
```

7.6 타일맵 설정

1. Mapviz 실행 후 좌측 하단 **add** 클릭
2. Select New Display 목록 최하단의 **tile_map** 선택 → OK
3. **Base URL** 에 입력:

```
http://localhost:8080/wmts/gm_layer/gm_grid/{level}/{x}/{y}.png
```

4. **Max Zoom** 을 19로 수정 (선택)
5. Save — 저장 이름을 **google map** 으로 하면 Source 목록에 그 이름으로 표시된다

8. MATLAB / Simulink ROS 2 연동

원본 자료에 없던 부분이다. 6주차 실습의 전제 조건이므로 반드시 확인할 것.

8.1 전제

- **MATLAB R2024b** (Windows 측에 설치)
- **ROS Toolbox** 설치 확인:

```
ver('ros')
```

- MATLAB R2024b의 ROS Toolbox는 **ROS 2 Humble**을 기본 지원한다. WSL의 ROS 2와 별도의 내부 ROS 2 런타임을 갖고 있으므로, 통신은 **DDS를 통해** 이루어진다.

8.2 네트워크 조건

- WSL2는 기본적으로 NAT 모드라 Windows와 WSL이 **다른 서버넷**에 있다. DDS 멀티캐스트 디스커버리가 이 경계를 넘지 못하는 경우가 있다.

해결 순서 (위에서부터 시도)

① WSL 미러 네트워크 모드 (Windows 11 22H2 이상, 가장 간단)

- Windows 쪽 **C:\Users\<사용자명>\.wslconfig** :

```
[wsl2]
networkingMode=mirrored
```

- PowerShell에서 **wsl --shutdown** 후 재시작. 이 모드에서는 WSL과 Windows가 같은 네트워크 인터페이스를 공유하므로 별도 설정 없이 통신된다.

② Domain ID 통일

- 양쪽 모두 같은 값을 쓴다. 실습실에서 여러 학생이 동시에 작업하므로 **팀 번호를 Domain ID로 사용**한다 (0~101 범위).
- WSL 쪽 (**~/.bashrc** 에 추가):

```
export ROS_DOMAIN_ID=7
```

MATLAB 쪽:

```
setenv("ROS_DOMAIN_ID", "7")
```

③ RMW 구현 통일

```
sudo apt install -y ros-humble-rmw-cyclonedds-cpp
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

```
setenv("RMW_IMPLEMENTATION", "rmw_cyclonedds_cpp")
```

8.3 통신 검증 절차

Step 1 — WSL 터미널에서 발행:

```
ros2 topic pub /test_topic std_msgs/msg/String "{data: 'hello from wsl'}" -r 1
```

Step 2 — MATLAB 명령창에서:

```
setenv("ROS_DOMAIN_ID", "7")
ros2 node list
ros2 topic list
```

- `/test_topic` 이 목록에 보이면 디스커버리 성공.

Step 3 — MATLAB에서 구독:

```
node = ros2node("/matlab_test");
sub = ros2subscriber(node, "/test_topic", "std_msgs/String");
msg = receive(sub, 10)
```

Step 4 — 역방향 확인. MATLAB에서 발행:

```
pub = ros2publisher(node, "/from_matlab", "std_msgs/String");
send(pub, ros2message(pub));
```

WSL에서:

```
ros2 topic echo /from_matlab
```

- 네 단계가 모두 통과하면 6주차 실습 준비가 끝난 것이다.

8.4 VRX와 직접 연동 확인

```
# WSL: VRX 실행
ros2 launch vrx_gz competition.launch.py world:=sydney_regatta
```

```
% MATLAB
node = ros2node("/vrx_check");
sub = ros2subscriber(node, "/wamv/sensors/gps/gps/fix", "sensor_msgs/NavSatFix");
msg = receive(sub, 10);
fprintf("lat=%.6f lon=%.6f\n", msg.latitude, msg.longitude);

% 추력 명령 발행
pub = ros2publisher(node, "/wamv/thrusters/left/thrust", "std_msgs/Float64");
m = ros2message(pub); m.data = 200;
send(pub, m);
```

- Gazebo에서 WAM-V가 움직이면 **양방향 연동 완료**다.

8.5 연동이 끝내 안 될 때 (대안 경로)

- ROS Toolbox 직접 통신이 실패하면 **Simulink 코드생성** → **ROS 2 노드 빌드** 방식으로 우회한다. 연구실 워크스페이스의 `src/vrx_control_thruster_waypoints/` 가 이 방식으로 만들어진 선례다.
- 1. Simulink 모델 → Configuration Parameters → Hardware Implementation → **Robot Operating System 2 (ROS 2)**
- 2. 빌드 → C++ 소스 생성 → WSL 워크스페이스로 복사
- 3. `colcon build` 후 `ros2 run` 으로 실행
- 이 경우 실시간 파라미터 튜닝은 못 하지만, 제어기 로직 자체는 동일하게 검증할 수 있다.

9. 배포 이미지로 설치하기 (권장 경로)

- 조교가 배포하는 `vrx-ubuntu2204.tar` 를 사용하면 §3~§7을 건너뛸 수 있다.

9.1 import

- PowerShell (관리자 권한 불필요):

```
# 저장 위치 생성
mkdir C:\WSL\vrx

# 배포 tar를 import (경로는 실제 다운로드 위치로)
wsl --import vrx-ubuntu2204 C:\WSL\vrx C:\Users\<사용자명>\Downloads\vrx-ubuntu2204.tar --version 2

# 실행
wsl -d vrx-ubuntu2204
```

9.2 기본 사용자 설정

- import한 배포판은 root로 로그인된다. 일반 사용자로 바꾸려면 배포판 안에서:

```
# /etc/wsl.conf 생성
cat <<'EOF' | sudo tee /etc/wsl.conf
[user]
default=vrx
EOF
```

- PowerShell에서 `wsl --shutdown` 후 다시 실행한다.

9.3 확인

```
source ~/vrx_ws/install/setup.bash
ros2 launch vrx_gz competition.launch.py world:=sydney_regatta
```

9.4 조교용 — 이미지 만드는 법

```
wsl --shutdown
wsl --export Ubuntu-22.04 D:\dist\vrx-ubuntu2204.tar
```

export 전에 `~/.bash_history` 정리, `apt clean`, 빌드 캐시(`build/`, `log/`) 삭제를 권장한다. 정리하면 8~10 GB 수준으로 줄어든다.

10. 원본 자료 대비 변경 이력

#	위치	원본	개정	사유
1	§1.1	<code>wsl --install -d Ubuntu-22.04</code>	<code>wsl --install -d Ubuntu-22.04</code>	유니코드 엔대시 → ASCII 하이픈. 원본 그대로는 실행 실패
2	§5.1	<code>git clone</code> 만	<code>git checkout humble</code> 추가	기본 브랜치는 Jazzy+Harmonic용. Garden 환경에서 빌드/실행 실패
3	§6.1	<code>ros-\$ROS_humble-mapviz</code>	<code>ros-humble-mapviz</code>	<code>\$ROS_DISTRO</code> 오용. 원본 슬라이드 18~19 설치 실패의 원인
4	§7.1	저장소 <code>bionic</code> , <code>apt-key add</code>	<code>\$(lsb_release -cs)</code> , keyring 파일 방식	Ubuntu 22.04는 <code>jammy</code> . <code>apt-key</code> 는 deprecated
5	§1.4	<code>Ctrl+Alt+E/O/W</code>	<code>Ctrl+Shift+E/O/W</code>	Ubuntu 22.04 terminator 기본 키맵
6	§2.1	—	<code>.wslconfig</code> 리소스 제한 추가	메모리 부족으로 인한 빌드·실행 실패 예방
7	§2.2	—	WSLg GUI 확인 절차 추가	원본은 GUI 동작을 전제만 하고 확인 절차가 없음
8	§7.3	—	WSL Docker 데몬 시작 절차 추가	WSL2는 systemd 기본 비활성
9	§8 전체	—	MATLAB ROS 2 연동 신규 작성	6주차 실습의 전제 조건인데 원본에 전무
10	§9 전체	—	WSL 이미지 배포/import 신규 작성	1주차 설치 실패 시간 손실 최소화

11. 자주 발생하는 문제

증상	원인	해결
<code>colcon build</code> 중 프로세스가 죽음	메모리 부족	<code>--parallel-workers 2</code> 추가, <code>.wslconfig</code> 로 메모리 상향
Gazebo 창이 안 뜨고 멈춤	WSLg 미작동	PowerShell <code>wsl --update</code> , §2.2 <code>xeyes</code> 확인
<code>ros2 topic list</code> 에 아무것도 없음	워크스페이스 미적용	<code>source ~/vrx_ws/install/setup.bash</code>
MATLAB에서 WSL 토픽이 안 보임	DDS 디스커버리 차단	§8.2 ① 미리 모드 → ② Domain ID → ③ RMW 순으로 시도
VRX 빌드 시 플러그인 오류	브랜치 미지정	<code>cd ~/vrx_ws/src/vrx && git checkout humble</code> 후 재빌드
Gazebo가 극도로 느림	GPU 미사용 / RAM 부족	파랑 렌더링 끄기, 다른 프로그램 종료, 워크스테이션 원격 사용
<code>docker: permission denied</code>	그룹 미적용	<code>sudo usermod -aG docker \$USER</code> 후 <code>wsl --shutdown</code>
빌드는 되는데 실행 시 모델을 못 찾음	환경변수 누락	새 터미널에서 <code>source ~/.bashrc</code> 확인